1 Decision Boundaries

1A)
(i) No. Because this is a 3-dimensional range and if all x are at 0, then it will fail the and function. Likewise the invalid point will sit at the origin. And because any points on the decision boundary are considered misclassified, the singular point of (0,0,0) will invalidate the decision boundary.

(ii) A non-zero offset is allowed and necessary. As long as the offset doesn't go past a certain threshold.
$\vec{\theta}$= [1,1,1] b = -0.5
1B)
(i) Can classify
Because the positive values sit at $\sqrt{2}$away from the origin and the negative sit at 1 unit away from the origin, the circle can differentiate between positive and negative values. If the outside of the circle is positive and has a radius between 1 & $\sqrt{2}$.
(ii) Can't classify
No classifier exists, because the data points are not linearly separable through the origin. The positive points sit on a distinct linear line through the origin so an offset is required to properly classify the points.
(iii) Can classify.
Outside the square with side length between 1 and 3 (non-inclusive). And [a,b]= [-0.5,0.5] to name one example.
(iv) Can classify.
Outside of a square with $\alpha = 45$ so that the square is slanted that its corners sit on the axes. Length of the sides are in between $\sqrt{2}$ and $2\sqrt{2}$.

2 Perceptron algorithm with offset

2A)

$$\vec{\theta} = \sum_{i=1}^{\alpha_i} (y^{(i)} \vec{x}^{(i)})$$

$$b = \sum_{i=1}^{\alpha_i} (y^{(i)})$$

2B)
Distance from the origin to the decision boundary is defined by b, the offset. This value b, is then normalized by dividing against the normal vector θ which returns the direct distance from the origin to the closest point onto the decision boundary. By dividing against ||theta|| also provides information of the sign/direction. This therefore gives the shortest signed distance.

2D)
Received a theta of [4,0]
b: -6
total misclassification of 18

```
Done!
============== Classifier ==============
Theta:  [4. 0.]
b:  -6.0


============== Alpha ===================
i        Number of Misclassifications
========================================
0                1.0
1                0.0
2                11.0
3                2.0
4                0.0
5                4.0
Total Number of Misclassifications:  18.0
```

2E)
Yes, order matters. If the correct cluster of points are first introduced into the algorithm, the algorithm will start with being more accurately aligned, and therefore lower the number of mistakes seen/made. Since the points are linearly separable, order also helps converge the algorithm towards the right position. If too much clustered data is given, the algorithm may not converge properly until new data is introduced, therefore spaced out and distinct data will help the algorithm converge better/quicker.

3 Linear Regression

3.1D)
I don't think I did iterations correctly

Algorithm	learning_rate	θ_0	θ_1	# iterations	Runtime (s)
GD	10^{-4}	0.3508736	0.03223228	353622	3.157835498
GD	10^{-3}	0.35588962	0.02328778	52628	0.466530487
GD	10^{-2}	0.35747598	0.02045902	6988	0.061498231 99999993
GD	10^{-1}	0.35797914	0.01956179	870	0.007705484 000000151
SGD	10^{-4}	0.35654037	0.02205316	578020	1.031552179 9999998

SGD	10^{-3}	0.35739838	0.01984877	74240	0.131484734 9999999
SGD	10^{-2}	0.35463138	0.01764725	6960	0.012411206 000000119
SGD	10^{-1}	0.31333058	-0.00974263	1980	0.003590407 9999999894
Closed form	-	0.35820909	0.01915174	-	0.000200066 99999997046

3.1E)

GD tends to have less iterations since it completely iterates through the whole data batch before updating theta. Whereas SGD updates theta for every data point it checks. It seems that overall SGD runtime is faster than that of GD when given similar learning rates. ie learning rate of 10^{-1} for both has a runtime of SGD = ~0.0036 and GD = ~0.0077. Meaning that SGD runtime is faster since ~0.004 < ~0.008.

The fastest runtime for SGD and GD are both with the learning rate of 10^{-1}.

The coefficients both tend to match the closed form, though do tend to get slightly off when the learning rate gets too small.

3.1F)

The runtime of closed-form is significantly faster SGD, especially when it has a very small learning rate of 10^{-4} (reaching a runtime of around 1 second).

Closest coefficients comes from SGD with a learning rate of 10^{-3}.

3.1G)

Proposed learning rate $\eta_k = \frac{1}{0.001k + 1}$

Since the learning rate of 10^{-3} performs the closest to the closed form coefficients, I wanted to also increase the speed of which it iterates through (in which 10^{-1} is the fastest performing learning rate) so I wanted the learning rate to start from 10^{-1} then get to 10^{-3}. The +1 is to prevent the learning rate from flagging out when there's 0 iterations.

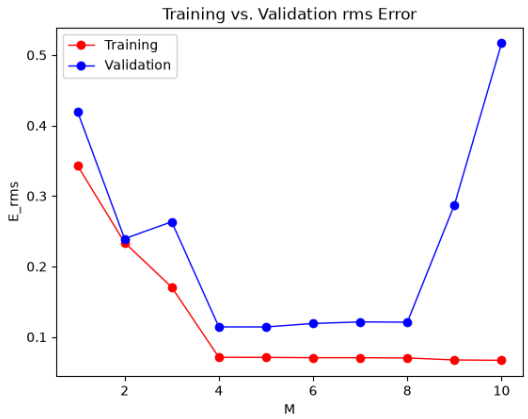
Time elapsed: 0.598851231 which is slower than all the constant learning rates except for the smallest of 10^{-4}.

Coefficients: [0.35740845 0.01829104]

The coefficients are relatively close to the closed form, with it performing slightly better in the first coefficient compared to that of the 10^{-3} learning rate, but worse for the second coefficient.

3.2 Linear Regression - Overfitting & Regularization

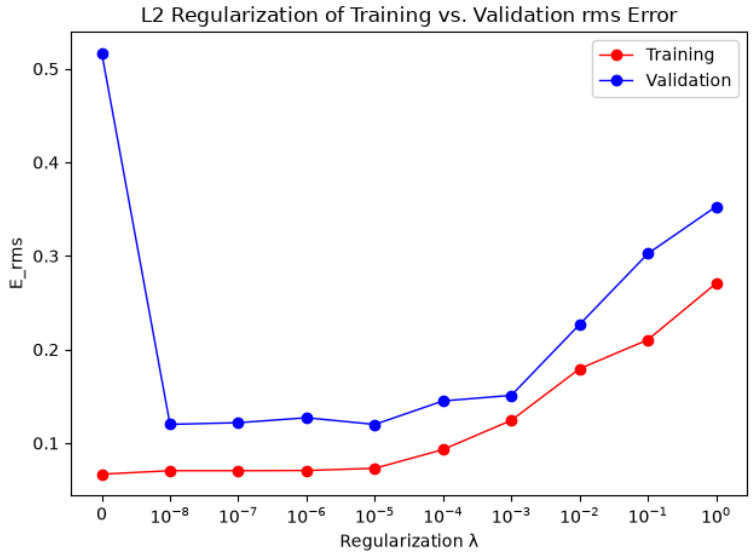
3.2B)



3.2C)

Degrees 4 through 8 best fits the data as the error is low for both training data & validation data. Underfitting occurs when there's high error for both data sets, since the model is not used to both sets, so around degrees 1 to 3 are underfitted. Overfitting occurs when the validation data has high error in comparison to the training data, because the model is built for the training data, but doesn't perform well on the actual validating data. Overfitted data will probably be on degrees 9 & 10.

3.2E)



3.2F)

$\lambda = 10^{-8}$ seems to be working the best. With the general range from around 10^{-8} to 10^{-5} working optimally. This is because both the training data and the validated data seem to have the lowest error compared to the other chosen lambda values.

3.2G)

I would probably model the data after the range of M (4,8) and λ ($10^{-8}, 10^{-5}$) because it seems to run well for both tests. However I would probably run various degrees (M) and lambda to get a better understanding of the relationship of the two values before modeling it after that. To be safe probably just model after degree 10 with a lambda somewhere along 10^{-8} to 10^{-5}. Since it already seems to perform well.

4 Weighted Linear Regression

4A)

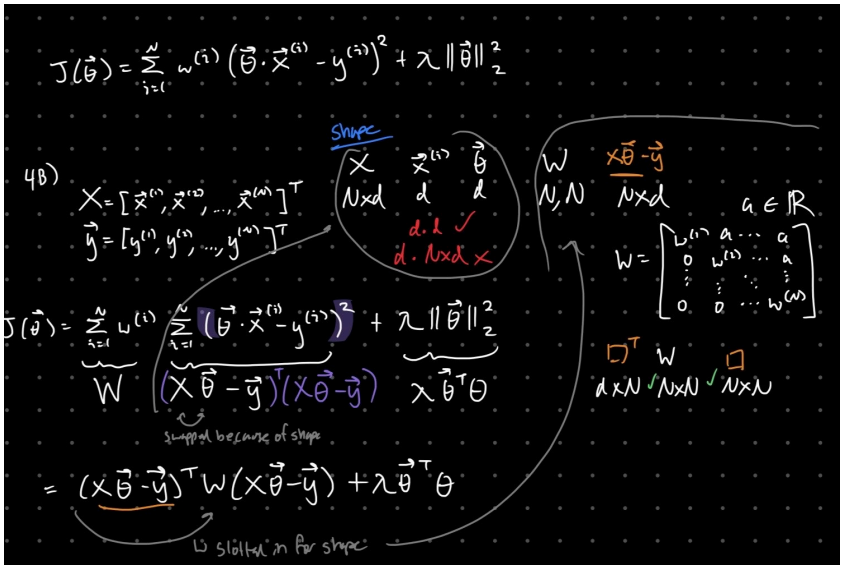
Weighted examples would be useful in data that has lots of noise or outliers. By being able to weigh accurate data more 'heavily' it will help focus the algorithm to learn better.

4B)

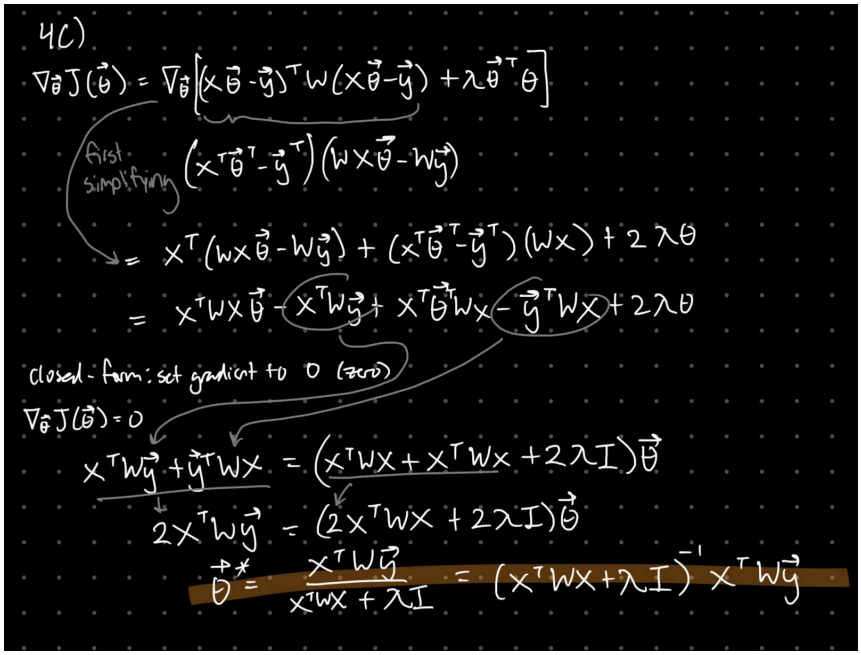
As established X represents the array containing x(i) vectors from x(i) to x(N), $\vec{y}$ contains the y(i) values and W is the diagonal weight matrix that contains the w(i) values.

Therefore the summation from 1 to N of $\vec{\theta} \cdot \vec{x}^{(i)} - y^{(i)}$ can be represented as $X\vec{\theta} - \vec{y}$. because it is squared there is 2 (one transposed to be able to multiply) and the w(i) is changed into a W resulting in the form: $(X\vec{\theta} - \vec{y})^T W(X\vec{\theta} - \vec{y})$. Likewise theta is also squared which results in the lambda times theta transposed and normal theta.

W is the diagonal weight matrix that contains the weight multiplier for each individual training example.



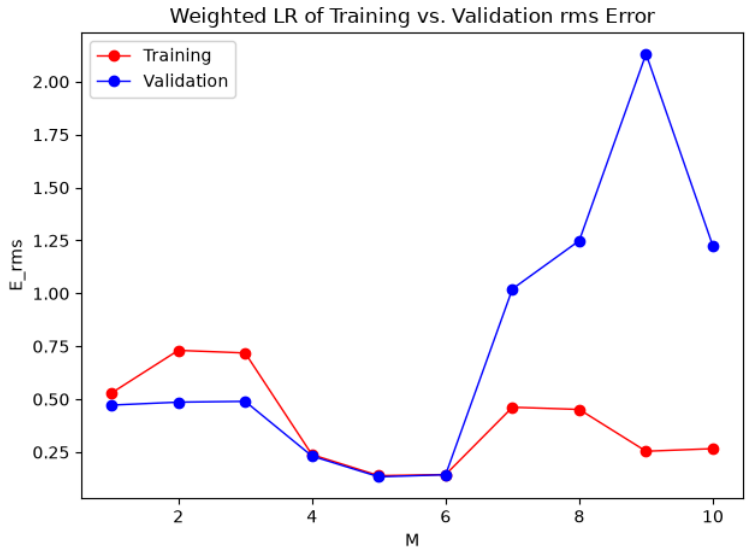
4C) $\vec{\theta} = (X^T W X + \lambda I)^{-1} X^T W \vec{y}$??



4D)??

```
# TODO: Implement this function
W = np.diag(weights)
tmp = np.linalg.inv(X.T @ W @ X + reg_param * np.eye(X.shape[1]))
theta = tmp @ X.T @ W @ y
return theta
```

4E)



Not the same conclusion, but generally the same. What is surprising is how even though the training data has plenty of errors, strangely the validated data actually performs better for

degrees lower than 4. Then similarly degrees 4 to 6 are equally low (also strangely seeming perfectly matching.) And the higher lambda's also overshoot, with the training having relatively low error despite not working for the validation data.
I think this occurs because of the integrated regularization within this function which helps balance the well fit and model complexity.